

Translated

From: e-maxx.ru

Generating all K -combinations

In this article we will discuss the problem of generating all K -combinations. Given the natural numbers N and K , and considering a set of numbers from 1 to N . The task is to derive all **subsets of size K** .

Generate next lexicographical K -combination

First we will generate them in lexicographical order. The algorithm for this is simple. The first combination will be $\{1, 2, \dots, K\}$. Now let's see how to find the combination that immediately follows this, lexicographically. To do so, we consider our current combination, and find the rightmost element that has not yet reached its highest possible value. Once finding this element, we increment it by 1 , and assign the lowest valid value to all subsequent elements.

```
bool next_combination(vector<int>& a, int n) {
    int k = (int)a.size();
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i + 1) {
            a[i]++;
            for (int j = i + 1; j < k; j++)
                a[j] = a[j - 1] + 1;
            return true;
        }
    }
    return false;
}
```

Generate all K -combinations such that adjacent combinations differ by one element

This time we want to generate all K -combinations in such an order, that adjacent combinations differ exactly by one element.

This can be solved using the [Gray Code](#): If we assign a bitmask to each subset, then by generating and iterating over these bitmasks with Gray codes, we can obtain our answer.